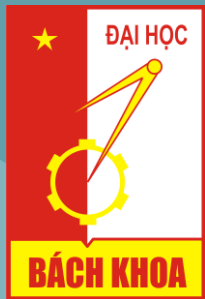


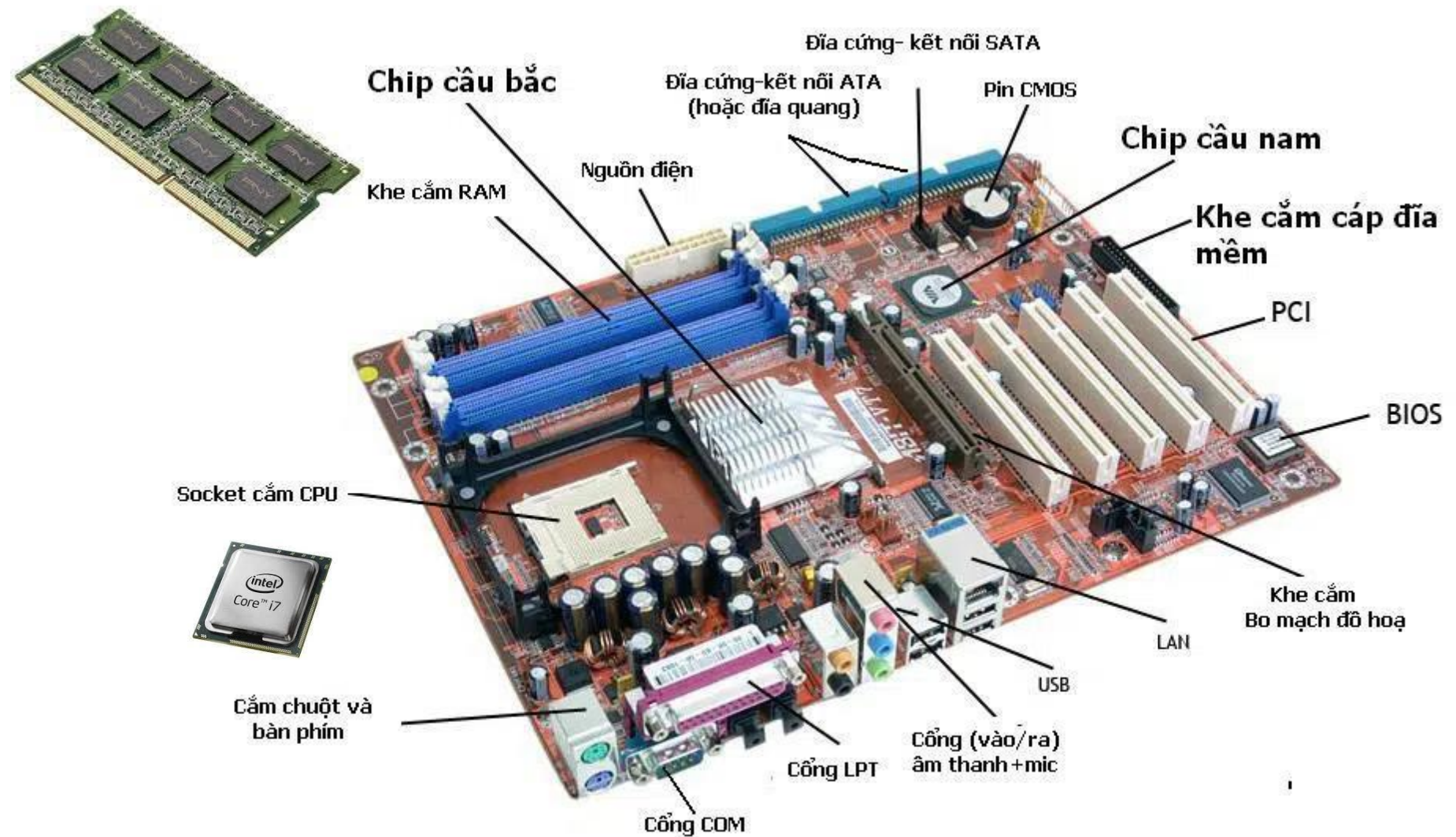
C PROGRAMMING INTRODUCTION



TUẦN 11: CON TRỎ

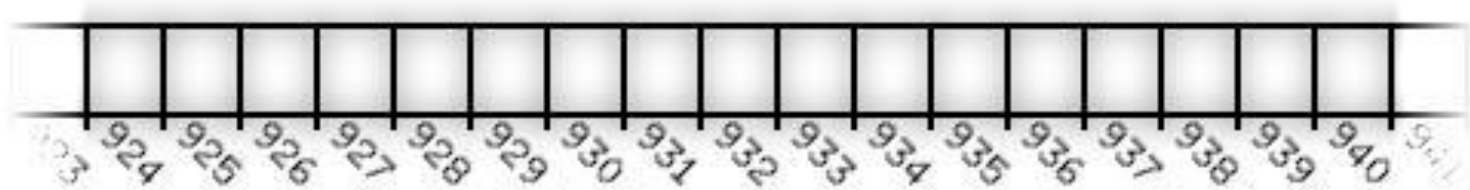
**Trường Công nghệ Thông tin và Truyền thông
Trường Đại học Bách Khoa Hà Nội**

Mainboard



Địa chỉ ô nhớ

- RAM: Random Access Memory. Hai thao tác chính: Read / Write
- Bộ nhớ RAM:
 - Chia thành các ô nhớ, mỗi ô nhớ lưu trữ một byte.
 - Mỗi ô nhớ có một địa chỉ đi kèm.
- Ví dụ : địa chỉ từ 924 tới 940

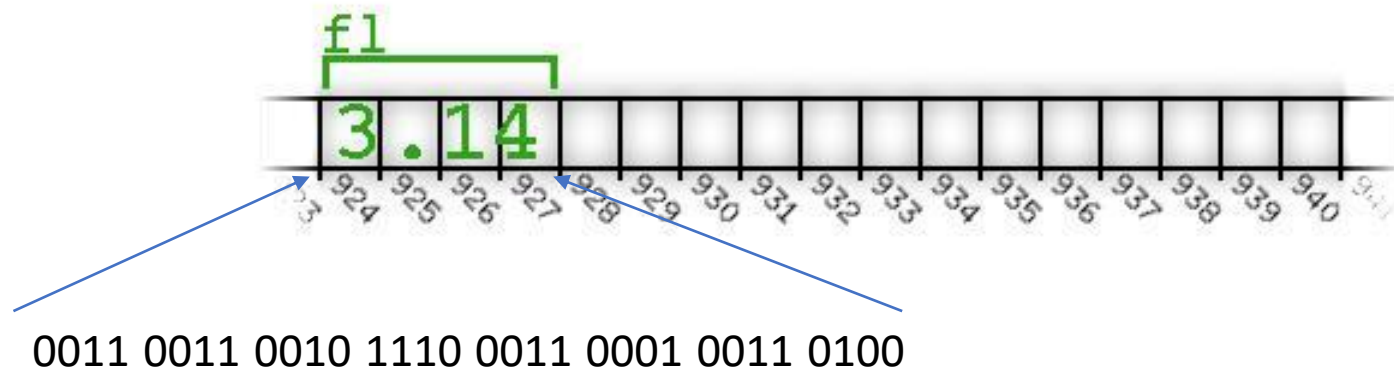


- 1 bộ nhớ RAM được đánh địa chỉ bởi 32 bit địa chỉ. Xác định dung lượng tối đa của bộ nhớ này?

Bài thực hành 1. Xác định địa chỉ ô nhớ của biến số thực

■ Toán tử **&** cho địa chỉ của một biến

```
#include <stdio.h>
int main() {
    float f1 = 3.14;
    printf("f1's address = %p\n", &f1);
    return 0;
}
```



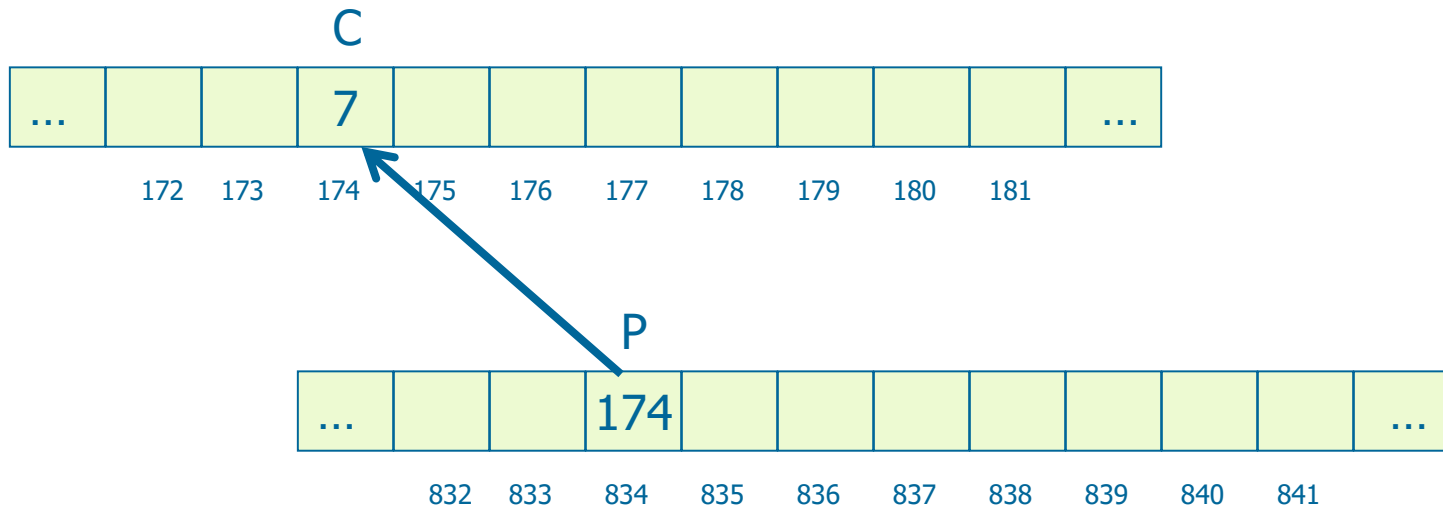
Khai báo một biến con trỏ (pointer)

```
type *variable_name;
```

- Con trỏ được khai báo bằng cách thêm * trước tên biến.
- Con trỏ là một biến có giá trị là một địa chỉ trong bộ nhớ: địa chỉ của biến hoặc mảng đã được khai báo.

Con trỏ

- ptr chứa địa chỉ của biến c
- ptr trỏ đến biến c

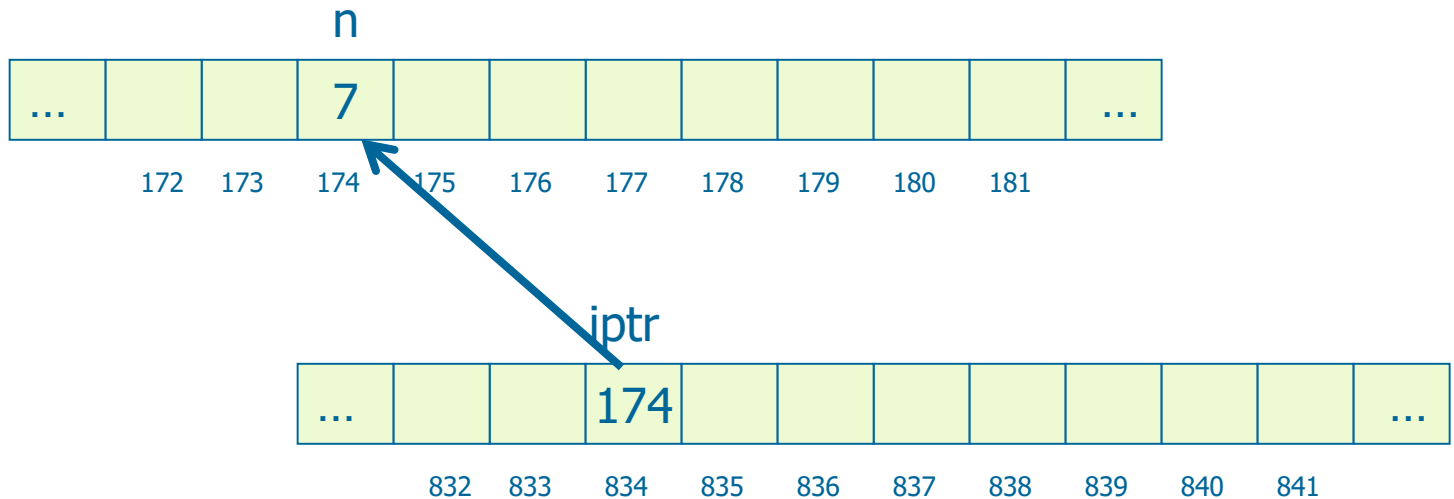


Toán tử địa chỉ

- Toán tử **&** cho địa chỉ của biến
- Câu lệnh: **ptr = &c;**
- gán địa chỉ của **c** tới biến con trỏ **ptr** : **ptr** trỏ tới **c**
- Để in một con trỏ, sử dụng định dạng **%p**.

Tham chiếu

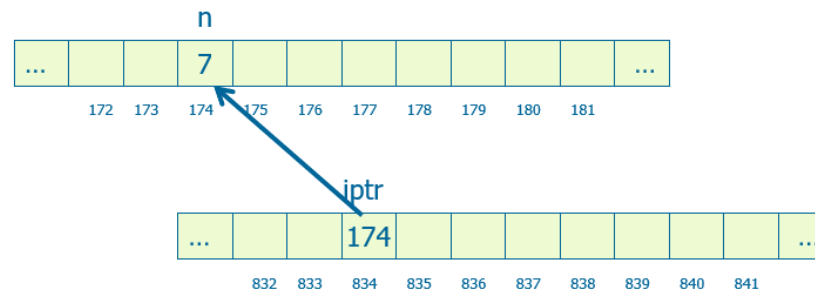
```
int n;  
int *iptr; /* Declare iptr as a pointer to int */  
n = 7;  
iptr = &n;
```



Khử tham chiếu

- Toán tử ***** dùng để khử tham chiếu
- Áp dụng trên các con trỏ
- Truy cập tới đối tượng mà con trỏ trỏ tới
- Câu lệnh: ***iptr = 5;**
gán vào **n** (biến mà **iptr** trỏ tới) giá trị 5

```
int n;  
int *iptr; /* Declare P as a pointer to int */  
n = 7;  
iptr = &n;
```



Bài thực hành 2

- Viết chương trình C sử dụng 3 biến để nhập 3 số nguyên
- Với mỗi biến, sử dụng một con trỏ để chỉ đến các biến này sau đó hiển thị giá trị của biến được trỏ đến bởi con trỏ
- Hiển thị địa chỉ của các biến x, y, z và nhận xét kết quả thu được. Vẽ bản đồ bộ nhớ có chứa các biến x, y, z này.

```
#include <stdio.h>
```

Lời giải

```
int main() {  
    int x, y, z;  
    int* ptr;  
    printf("Enter three integers: ");  
    scanf("%d%d%d", &x, &y, &z);  
    printf("\nThe three integers are:\n");  
    ptr = &x;  
    printf("x = %d\n", *ptr);  
    ptr = &y;  
    printf("y = %d\n", *ptr);  
    ptr = &z;  
    printf("z = %d\n", *ptr);  
    return 0;  
}
```

Bài thực hành 3

- Viết chương trình yêu cầu người dùng nhập vào 3 số thực x, y, z.
- Thêm 100 vào 3 số chỉ sử dụng một con trỏ ptr
- Các biến được khai báo như sau:

```
int x = 25, y = 50, z = 75;  
int *ptr;
```

Bài thực hành 4

Hàm swap lỗi

Hàm swap nhận **giá trị** các biến và **không** thay đổi giá trị của các biến gốc

```
void swap(int x, int y){  
    int tmp = x;  
    x = y;  
    y = tmp;  
}
```

Truyền tham số bởi giá trị

Bài thực hành 4 - HUSTack

Sửa lỗi

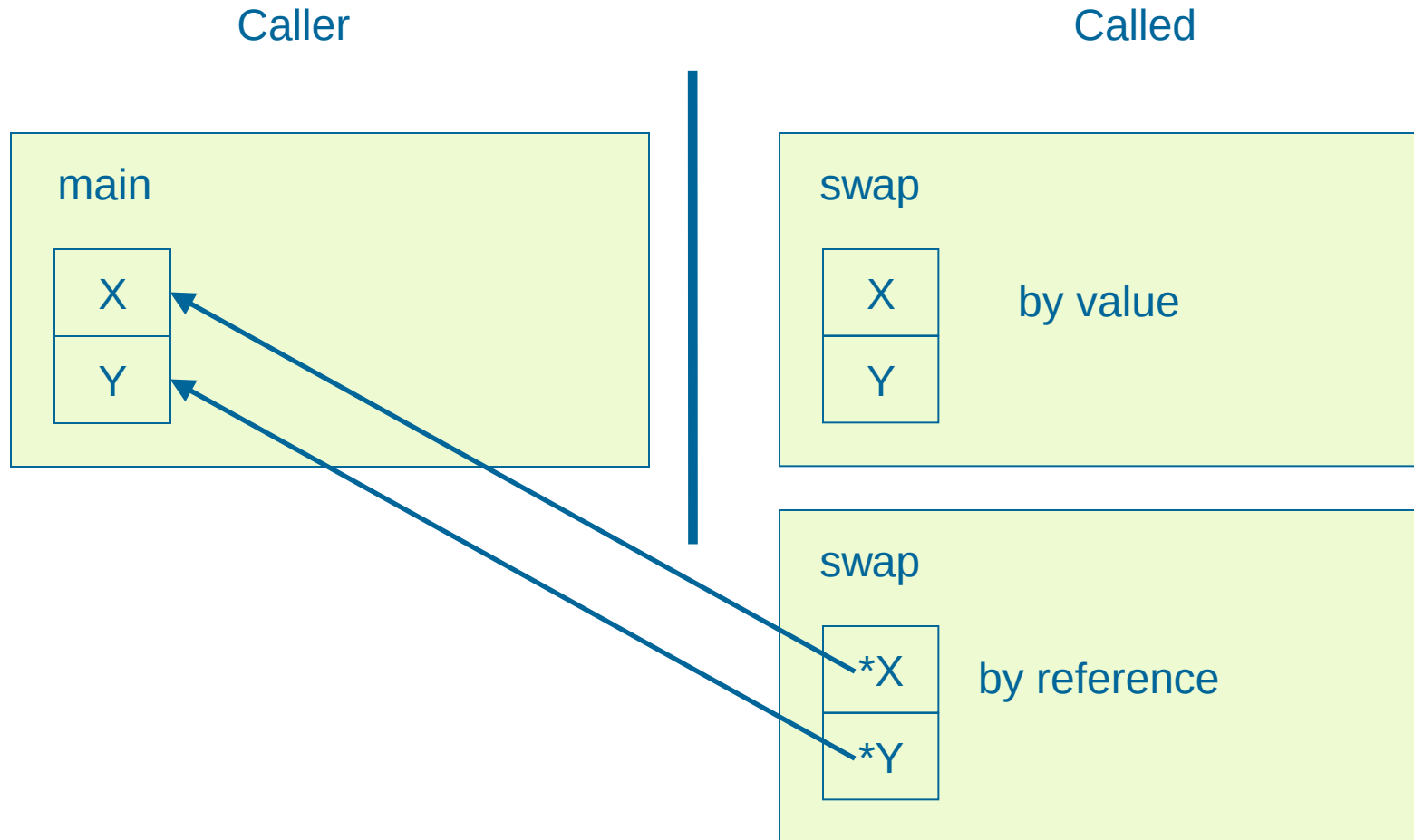
Viết chương trình thử nghiệm hai hàm swap

- Cho hàm swap nhận con trỏ tới các số nguyên

```
void swap(int *px, int *py)
{
    int temp = *px;
    *px = *py;
    *py = temp;
}
```

- Gọi hàm **swap (&x, &y) ;**
- Truyền tham chiếu

Truyền tham số bởi con trỏ



Bài thực hành 5 - HUSTack

- Viết hàm nhận 3 biến a, b, c và đổi chỗ sao cho giá trị của a thành b, b thành c và c thành a.
- Cú pháp của hàm:

```
void swap3(int *a, int *b, int *c);
```

- Kiểm tra hàm này bằng một chương trình: hiển thị giá trị a, b, c trước và sau khi truyền vào hàm

Bài thực hành 6

Khai báo các biến số nguyên **x**, **y**, **z** và các con trỏ số nguyên **p**, **q**, **r**. Khởi tạo giá trị cho **x**, **y**, **z**.

Khởi tạo **p**, **q**, **r** tới địa chỉ của **x**, **y**, **z**.

- 1) In ra giá trị của **x**, **y**, **z**, **p**, **q**, **r**, ***p**, ***q**, ***r**.
- 2) Đổi chỗ các giá trị **x**, **y**, **z**. In ra giá trị của **x**, **y**, **z**, **p**, **q**, **r**, ***p**, ***q**, ***r**.
- 3) Đổi chỗ giá trị **p**, **q**, **r**. In ra giá trị của **x**, **y**, **z**, **p**, **q**, **r**, ***p**, ***q**, ***r**.

Câu 2 và 3 chạy độc lập với nhau.

- 4) So sánh các kết quả của các câu 1, 2, 3 và nhận xét.

Bài thực hành 7 - HUSTack

- Viết hàm *incomeplus* để nâng lương 3000000 cho nhân viên với điều kiện số năm làm việc > 3 năm
void incomeplus(long *current, int year)
- Kiểm tra hàm với một chương trình

Bài thực hành 7b - HUSTack

- Viết hàm *count* để đếm số ước số của số nguyên dương *n*, hàm *count* có cú pháp như sau:

`void count(int n, int *nb)`

- Với số ước số sẽ lưu vào biến trả bởi con trỏ *nb*.
- Lưu ý:
 - 1 cũng được coi là ước số của số nguyên
 - Không tính đến giá trị của *n*
- Kiểm tra hàm với một chương trình

Bài thực hành 8

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
# include <stdio.h>
void fun(int *ptr)
{
    *ptr = 30;
}
int main()
{
    int y = 20;
    fun(&y);
    printf("%d", y);

    return 0;
}
```

Bài thực hành 9

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
#include <stdio.h>

int main()
{
    int *ptr;
    int x;
    ptr = &x;
    *ptr = 0;
    printf(" x = %d\n", x);
    printf(" *ptr = %d \n", *ptr);
    *ptr += 5;
    printf(" x = %d\n", x);
    printf(" *ptr = %d\n", *ptr);
    (*ptr)++;
    printf(" x = %d\n", x);
    printf(" *ptr = %d\n", *ptr);
    return 0;
}
```

Bài thực hành 10

HUSTack

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
#include <stdio.h>
int main()
{
    int a;
    char *x;
    x = (char *) &a;
    a = 512;
    x[0] = 1;
    x[1] = 2;
    printf("%d\n", a);
    return 0;
}
```

Bài thực hành 11

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
#include<stdio.h>
void f(int *p, int *q)
{
    p = q;
    *p = 2;
}
int i = 0, j = 1;
int main()
{
    f(&i, &j);
    printf("%d %d\n", i, j);
    return 0;
}
```

Bài thực hành 12

```
int f(int x, int *py, int **ppz)
{
    int y, z;

    **ppz += 1;

    z = **ppz;

    *py += 2;

    y = *py;

    x += 3;

    return x + y + z;
}
```

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
int main()
{
    int c, *b, **a;
    c = 4;
    b = &c;
    a = &b;
    printf("%d ", f(c, b, a));
    return 0;
}
```

Bài thực hành 13

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

```
#include <stdio.h>
```

```
#define print(x) printf("%d ", x)
```

```
int x;
```

```
void Q(int z)
```

```
{
```

```
    z += x;
```

```
    print(z);
```

```
}
```

```
void P(int *y)
```

```
{
```

```
    int x = *y + 2;
```

```
    Q(x);
```

```
    *y = x - 1;
```

```
    print(x);
```

```
}
```

```
int main()
```

```
{
```

```
    x = 5;
```

```
    P(&x);
```

```
    print(x);
```

```
    return 0;
```

```
}
```

Bài thực hành 14

- Xác định kết quả của chương trình sau
- Giải thích kết quả thu được kèm theo chú thích (chú thích dòng bắt đầu bằng //) cho các câu lệnh quan trọng của chương trình

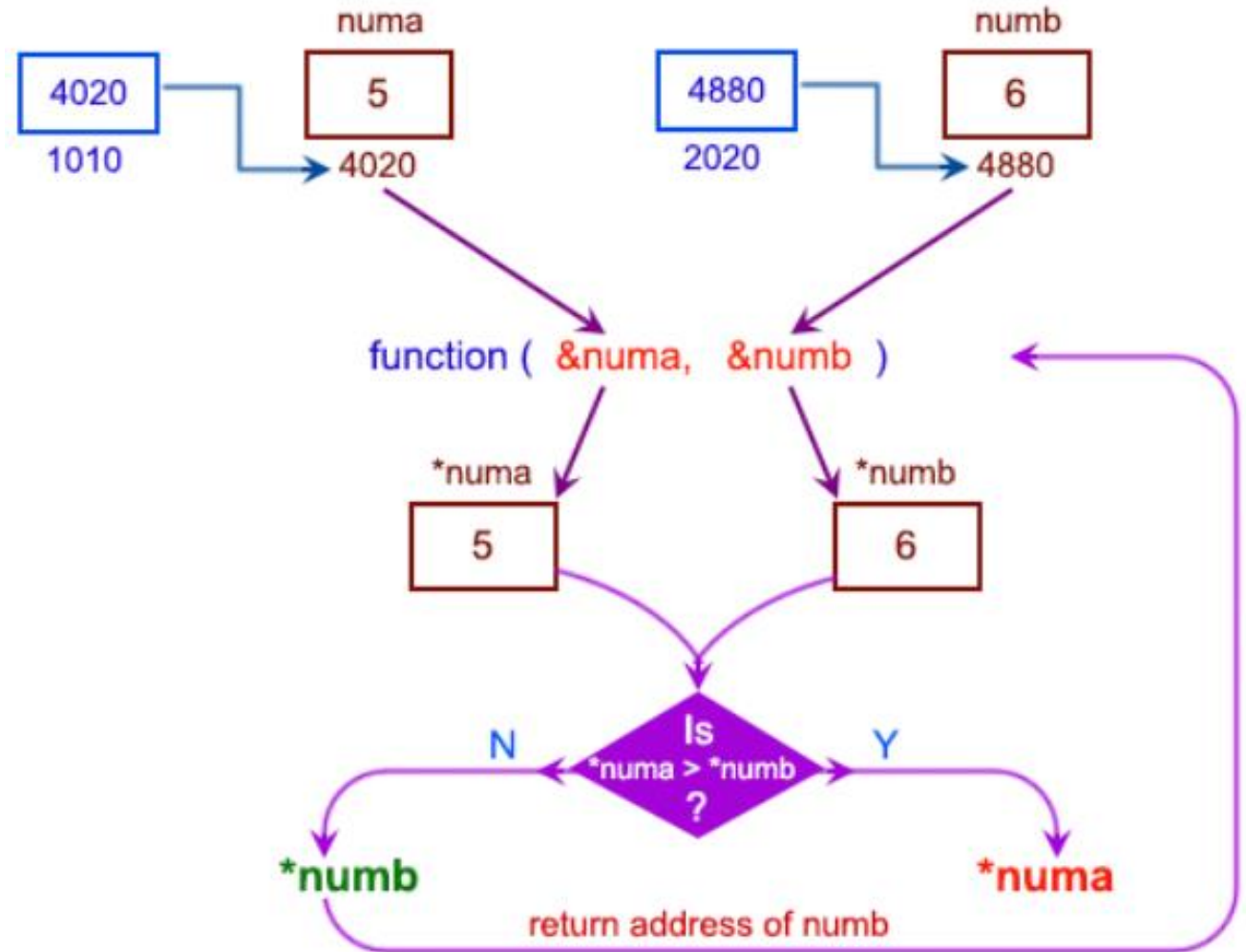
```
void f(int* p, int m)
{
    m = m + 5;
    *p = *p + m;
    return;
}
```

```
void main()
{
    int i=5, j=10;
    f(&i, j);
    printf("%d", i+j);
}
```

Bài thực hành 15a

Hàm trả về kiểu con trỏ

- Tìm số lớn nhất của hai số



Bài thực hành 15

```
#include <stdio.h>

int* findLarger(int *n1, int *n2)
{
    if(*n1 > *n2)
        return n1;
    else
        return n2;
}
```

Bài thực hành 15

```
int main()
{
    int numa = 0;
    int numb = 0;
    int *result;
    printf("\n\n Pointer : Show a function returning pointer
:\n");
    printf("-----\n");
    printf(" Input the first number : ");
    scanf("%d", &numa);
    printf(" Input the second  number : ");
    scanf("%d", &numb);
    result = findLarger(&numa, &numb);
    printf(" The number %d is larger. \n\n", *result);
    return 0;
}
```

Bài thực hành 15b - HUSTack

- Tìm số lớn nhất của ba số a, b, c
- Viết hàm có cú pháp như sau

`int* findLarger(int *n1, int *n2, int *n3)`

Bài thực hành 16

- Viết chương trình in ra các ký tự từ 'A' đến 'Z' sử dụng kiểu con trỏ

Bài thực hành 16 – Mã nguồn

```
#include <stdio.h>
int main() {
    char ch;
    char* pCh;
    pCh = &ch;
    for(ch = 'A'; ch <= 'Z'; ch++) {
        printf("%c ", *pCh);
    }

    return 0;
}
```

Bài thực hành 17 - HUSTack

- Viết hàm giải phương trình bậc một
- Hàm trả về giá trị là một số nguyên như sau:
 - 0: phương trình có nghiệm duy nhất $x = -b/a$
 - 1: phương trình vô số nghiệm
 - 2: phương trình vô nghiệm
- Cú pháp của hàm như sau:
`int gptb1(float a, float b, *float x);`

Bài thực hành 18 - HUSTack

- Viết hàm giải phương trình bậc hai
- Hàm trả về giá trị là một số nguyên như sau:
 - 0: phương trình có nghiệm duy nhất (do $\Delta = 0$)
 - 1: phương trình có hai nghiệm thực phân biệt ($\Delta > 0$)
 - 2: phương trình vô nghiệm ($\Delta < 0$)
 - 3: phương trình suy biến thành phương trình bậc 1 (do hệ số $a = 0$)
- Cú pháp của hàm như sau:
`int gptb1(float a, float b, float c, *float x1, *float x2);`